



MGA802 – Introduction à la programmation avec Python

Mini-Projet B

Membres de l'équipe

Nada Chaouachi | Asma Brik | Brahim Rezgui

1 Introduction

L'objectif de ce projet est d'étudier et de comparer différentes méthodes numériques d'intégration permettant de calculer l'intégrale d'un polynôme de degré trois.. Les résultats obtenus sont comparés à la solution analytique afin d'évaluer la précision de chaque méthode ainsi que leur temps d'exécution.

Les méthodes étudiées sont :

- La méthode des rectangles ;
- La méthode des trapèzes ;
- La méthode de Simpson ;
- Les implémentations vectorisées utilisant NumPy ;
- Les implémentations préprogrammées fournies par SciPy.

2 Structure du programme

Le programme a été développé sous forme modulaire afin de séparer les différentes responsabilités et faciliter la maintenance du code.

TABLE 1 – Organisation des modules

Module	Rôle
main.py	Point d'entrée du programme
integration_numerique.py	Calcul de la solution analytique et des erreurs
methode_rectangles.py	Implémentation de la méthode des rectangles
methodes_trapezes.py	Implémentation de la méthode des trapèzes
methodes_preprogrammees_trapezes.py	Utilisation des fonctions préprogrammées
methode_simpson.py	Implémentation de la méthode de Simpson (Python et NumPy)
methode_simpson_preprog.py	Simpson préprogrammée avec SciPy
graphiques.py	Génération des graphiques

Les bibliothèques principales utilisées sont :

- NumPy pour les calculs vectorisés ;
- Matplotlib pour les visualisations ;
- SciPy pour les méthodes d'intégration préprogrammées ;
- Timeit pour les mesures de performance.

3 Résultats

3.1 Résultats obtenus pour $n = 10$

TABLE 2 – Comparaison des méthodes pour $n = 10$

Méthode	Version	Valeur	Erreur	Temps (μs)
Exacte	Python	110	0	1.37
Rectangles	Python	107.03	2.97	9.60
	NumPy	107.03	2.97	58.80
Trapèzes	Python	111.875	1.875	5.45
	NumPy	111.875	1.875	21.84
	Préprogrammée	111.875	1.875	34.85
Simpson	Python	110.0000	0.0000	4.27
	NumPy	110.0000	0.0000	22.57
	Préprogrammée	110.0000	0.0000	21.82

Les simulations ont été réalisées pour :

$$n = 10, 20, 50, 100, 200, 500, 1000$$

Les graphiques permettent d'étudier la convergence des méthodes ainsi que l'évolution de l'erreur lorsque le nombre de segments augmente.

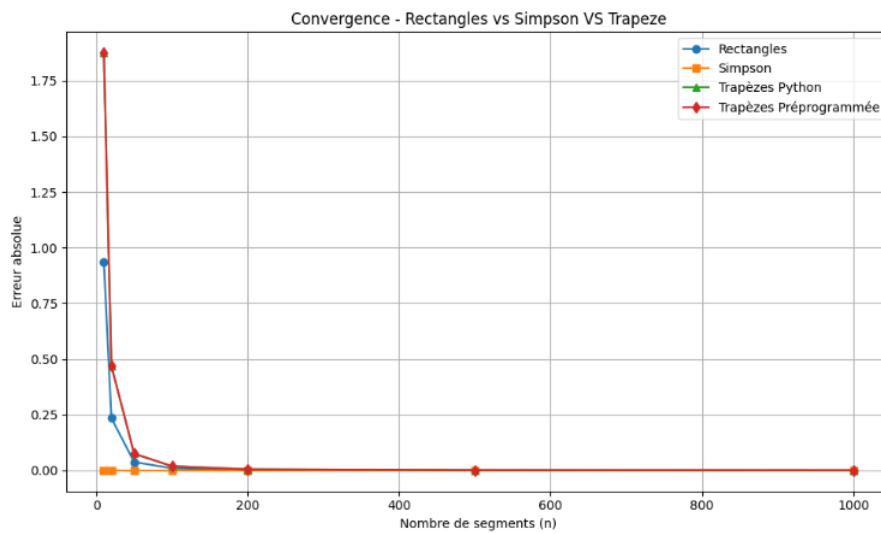


FIGURE 1 – Convergence des méthodes numériques

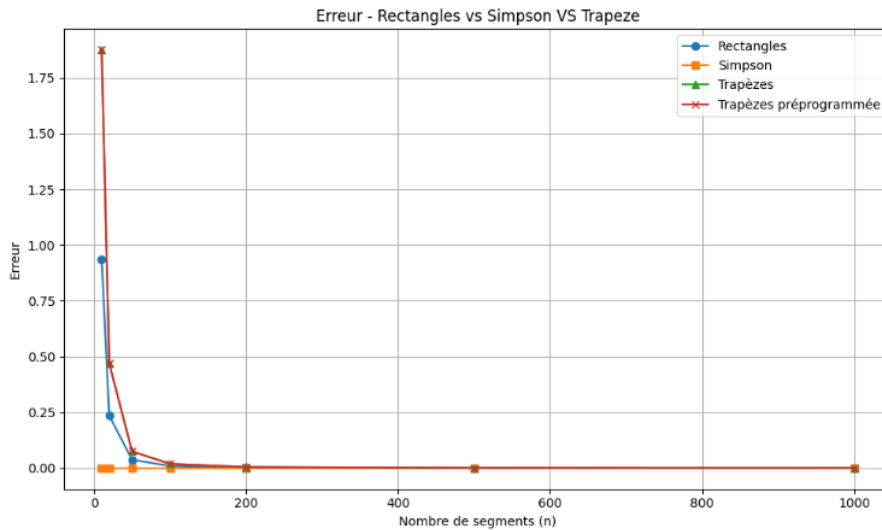


FIGURE 2 – Erreur en fonction du nombre de segments

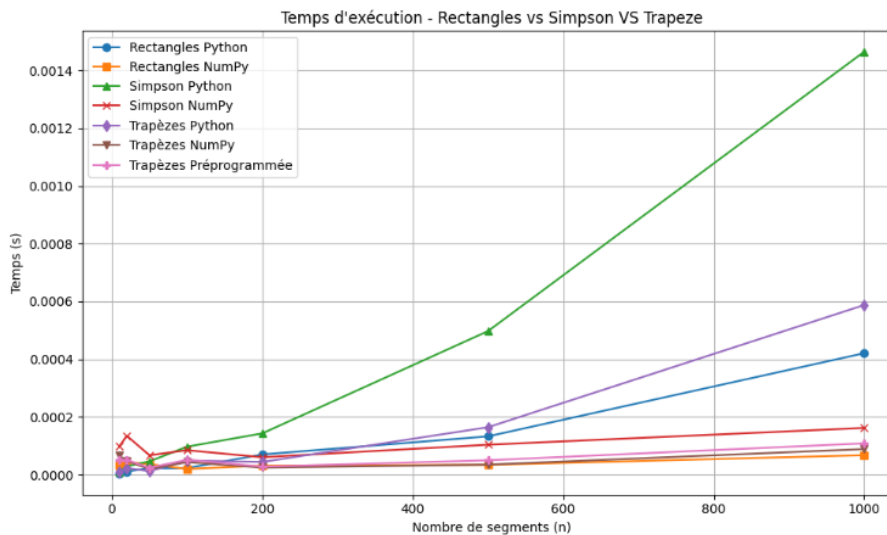


FIGURE 3 – Temps d'exécution par nombre de segments

4 Analyse des résultats

4.1 Méthode des rectangles

La méthode des rectangles approxime l'intégrale par une somme d'aires de rectangles. Dans notre implémentation, l'évaluation est réalisée au centre des sous-intervalles afin de réduire l'erreur numérique.

Pour $n = 10$, l'erreur obtenue est de 2,97. Les résultats montrent que l'erreur diminue lorsque le nombre de segments augmente. La version Python est plus rapide pour les petites tailles de problème, tandis que NumPy devient avantageux lorsque le nombre de segments est élevé grâce à la vectorisation.

4.2 Méthode des trapèzes

Les versions Python, NumPy et préprogrammée produisent exactement les mêmes résultats numériques. Cela confirme que les trois implémentations appliquent la même formulation mathématique.

Les courbes de convergence montrent une diminution régulière de l'erreur lorsque le nombre de segments augmente. Pour les faibles valeurs de n , l'implémentation Python présente le meilleur temps

d'exécution. Lorsque n devient plus important, les bénéfices de la vectorisation apparaissent progressivement.

4.3 Méthode de Simpson

La méthode de Simpson repose sur une interpolation quadratique entre les points d'intégration. Elle est généralement plus précise que les méthodes des rectangles et des trapèzes.

Les trois implémentations étudiées (Python, NumPy et SciPy préprogrammée) produisent pratiquement la même valeur numérique. Pour le polynôme de degré trois utilisé dans ce projet, la méthode de Simpson retrouve exactement la solution analytique, ce qui conduit à une erreur nulle.

Les courbes de convergence montrent que cette méthode atteint rapidement la valeur exacte même avec un faible nombre de segments. Les temps d'exécution mesurés demeurent très faibles pour les trois versions.

La version Python obtient le meilleur temps d'exécution pour les faibles valeurs de n , tandis que les versions NumPy et SciPy offrent une implémentation plus compacte et plus adaptée aux problèmes de grande taille.

La méthode de Simpson est donc la plus précise parmi toutes les méthodes étudiées dans ce projet.

5 Conclusion

Ce projet a permis de comparer plusieurs méthodes numériques d'intégration en termes de précision et de temps d'exécution.

Les résultats montrent que toutes les méthodes convergent vers la solution exacte lorsque le nombre de segments augmente. La méthode des rectangles est la moins précise, tandis que la méthode des trapèzes améliore significativement les résultats.

La méthode de Simpson s'est révélée être la plus performante du point de vue de la précision. Les versions Python, NumPy et SciPy ont toutes permis d'obtenir exactement la valeur analytique du polynôme étudié.

Enfin, les implémentations utilisant NumPy et SciPy facilitent le développement du programme tout en conservant d'excellentes performances numériques.